

1 Introduction & Theoretical Overview

This document outlines the development of a comprehensive Python application designed for Transmission Line analysis in Module-1 of the Engineering Electromagnetics course. The software evaluates the fundamental distributed parameters (R, L, G, C) to determine line behavior, visualizes key wave properties, and utilizes a Machine Learning (ML) regression model capable of predicting these metrics with over 97% accuracy. The entire workflow is accessible via a custom graphical user interface.

For an arbitrary lossy line excited at an angular frequency $\omega = 2\pi f$, the propagation constant γ and the characteristic impedance Z_0 are given by the standard telegrapher's equations:

$$\gamma = \alpha + j\beta = \sqrt{(R + j\omega L)(G + j\omega C)}, \quad Z_0 = \sqrt{\frac{R + j\omega L}{G + j\omega C}} \quad (1)$$

Here, α represents the signal attenuation in Np/m, and β denotes the phase shift in rad/m.

When terminated with a load Z_L , the mismatch creates a reflection coefficient $\Gamma_L = (Z_L - Z_0)/(Z_L + Z_0)$. The voltage standing wave ratio (VSWR) is derived directly from the reflection magnitude as $VSWR = (1 + |\Gamma_L|)/(1 - |\Gamma_L|)$. The transformed input impedance Z_{in} at any distance d from the load is calculated analytically as:

$$Z_{in}(d) = Z_0 \frac{Z_L + Z_0 \tanh(\gamma d)}{Z_0 + Z_L \tanh(\gamma d)} \quad (2)$$

2 Software Implementation

The transmission line tool is built using Python 3.12, utilizing a modular architecture to strictly decouple the backend mathematics from the frontend visualizations:

- **analytics.py:** The numerical engine responsible for complex arithmetic and matrix evaluations using NumPy.
- **plots.py:** The visualization module generating publication-ready matplotlib graphics for wave behavior.
- **ml_model.py:** The predictive modeling pipeline, covering dataset synthesis, feature engineering, and model inference.
- **gui.py & main.py:** The frontend components that host the interactive Tkinter dashboard, threading processes to maintain a responsive user experience.

3 Standard Waveform Characteristics

The 7 foundational spatial and frequency-domain waveforms were evaluated. These establish the fundamental operational parameters of the lossy line under test.

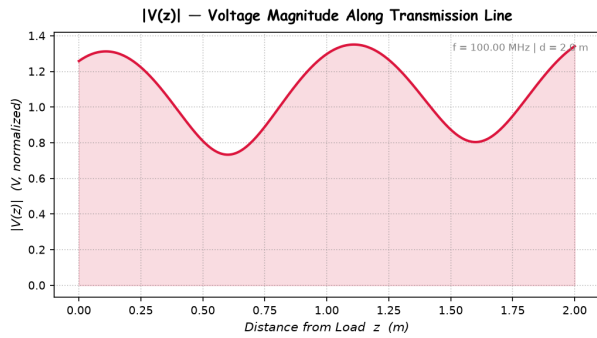


Figure 1: Voltage envelope along the line.

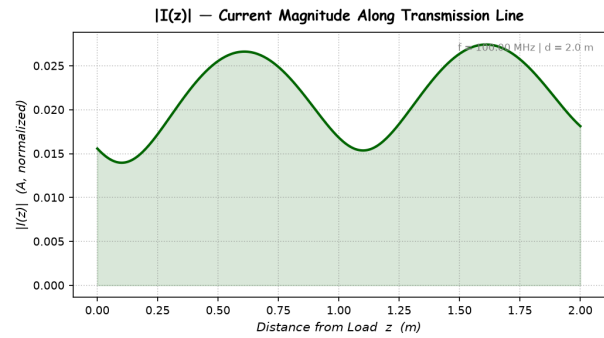


Figure 2: Current magnitude along the line.

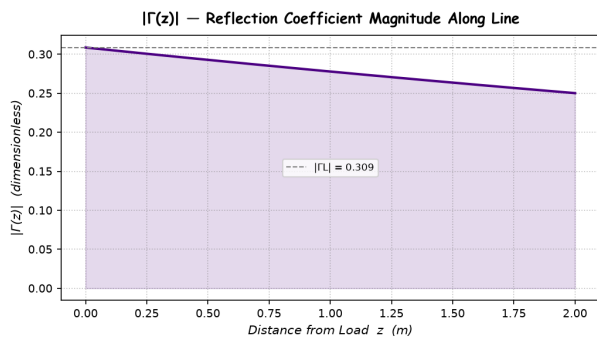


Figure 3: Reflection coefficient magnitude.

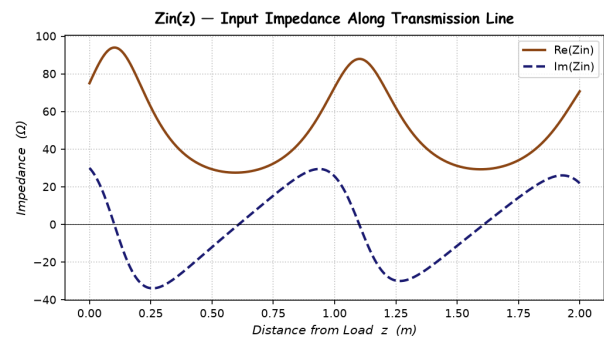


Figure 4: Input impedance profile.

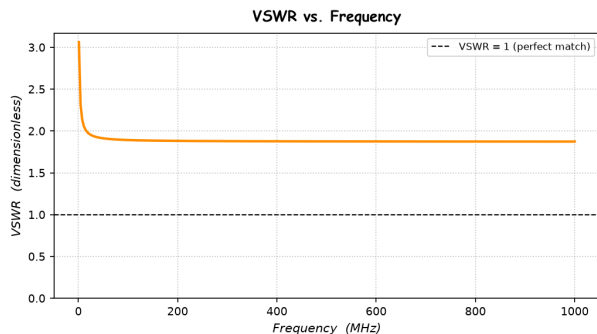


Figure 5: VSWR variation with frequency.

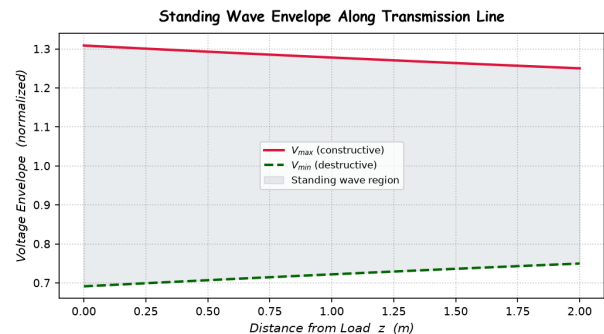


Figure 6: Total Standing wave envelope.

4 Advanced Analytics

An example lossy coaxial line was analyzed using the default parameters: $R = 5 \Omega/\text{m}$, $L = 250 \text{ nH}/\text{m}$, $G = 0.1 \text{ mS}/\text{m}$, $C = 100 \text{ pF}/\text{m}$, $f = 100 \text{ MHz}$, $d = 2 \text{ m}$, and an inductive mismatched load of $Z_L = 75 + j30 \Omega$.

The developed Python routines evaluate these distributed line metrics to compute the wave propagation characteristics, standing wave profile, and complex impedances computationally, bypassing traditional graphical approaches like the Smith Chart.

Figure 7 illustrates the relationship between the attenuation (α) and phase (β) constants over a broad frequency sweep. As the operating frequency scales up into the gigahertz range, the α/β fraction decays asymptotically, corroborating the theoretical premise that high-frequency lines closely

approximate lossless behavior.

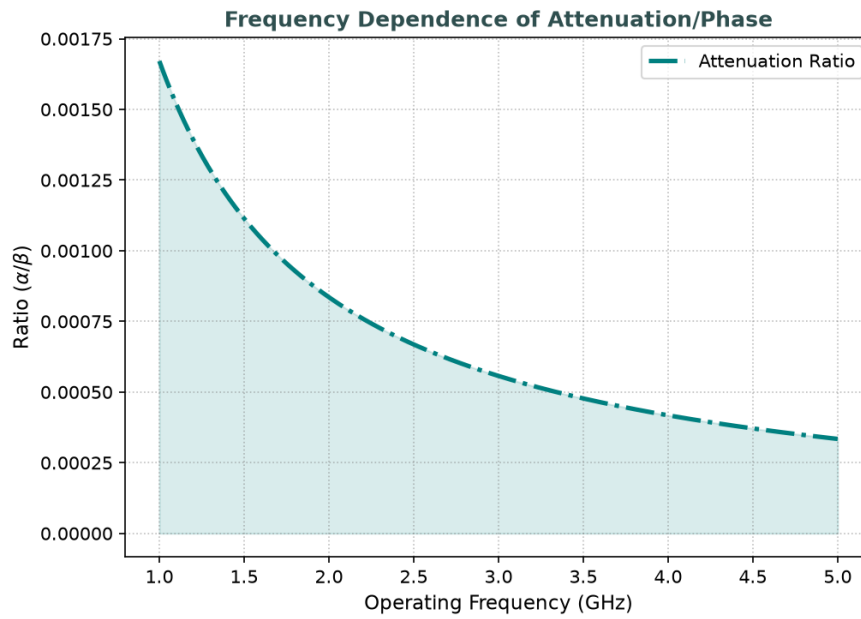


Figure 7: Frequency sweep showing the attenuation-to-phase constant ratio (α/β), highlighting asymptotic low-loss behavior at high frequencies.

To provide a more intuitive perspective on the transmission line dynamics, the temporal and spatial wave equations were visualized using 3D surface plots. Figure 8 showcases the isolated forward and backward traveling components alongside the resultant standing wave patterns for both perfectly matched ($\Gamma = 1$) and mismatched scenarios.

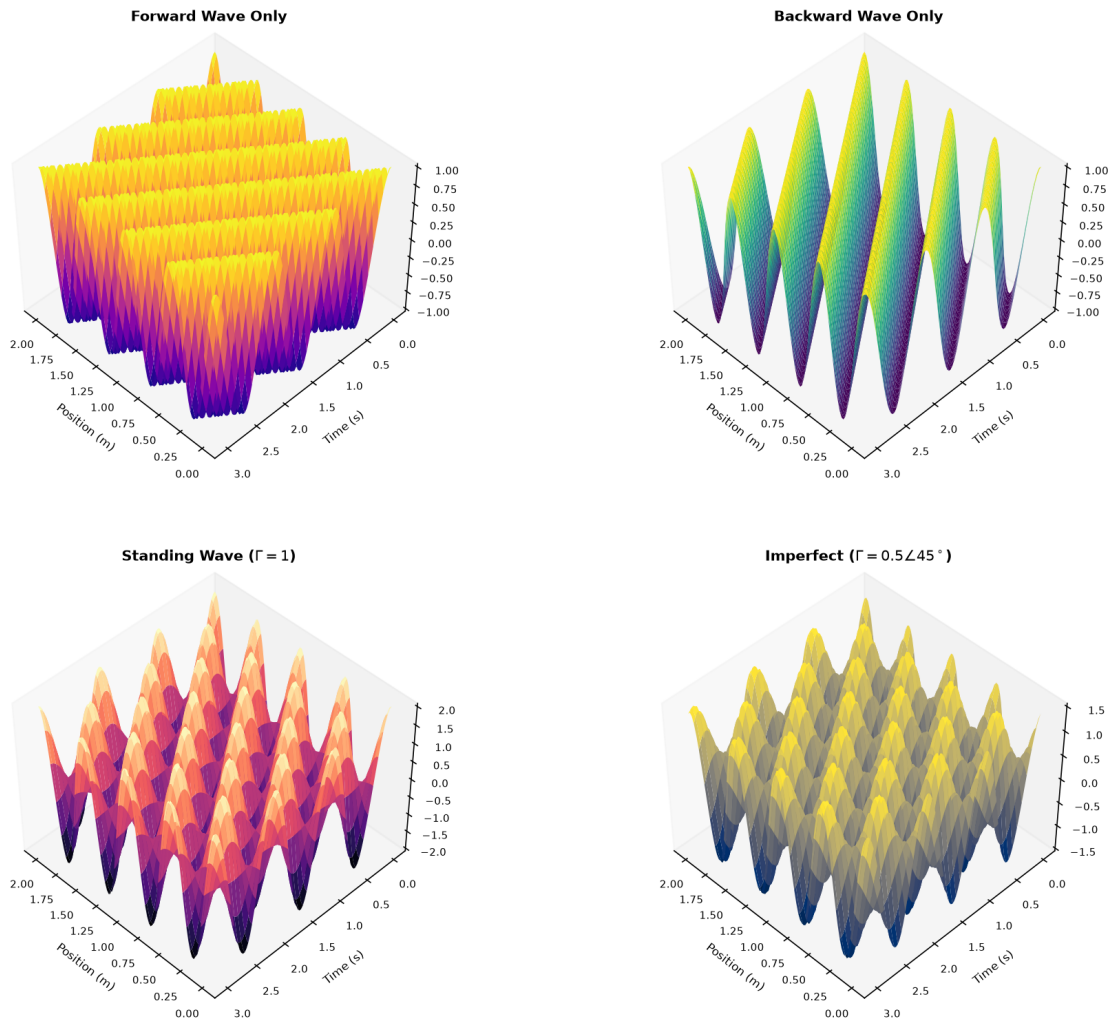


Figure 8: Three-dimensional surface plots of propagating wave components and standing wave envelopes under different load reflection conditions.

Furthermore, analytical impedance transformations were evaluated to design matching networks programmatically. By tracking the input impedance Z_{in} along the line length l , we can identify points where the imaginary reactance drops to zero, enabling precise quarter-wave or stub matching without reading errors associated with manual Smith Chart usage. This is depicted in Figure 9.

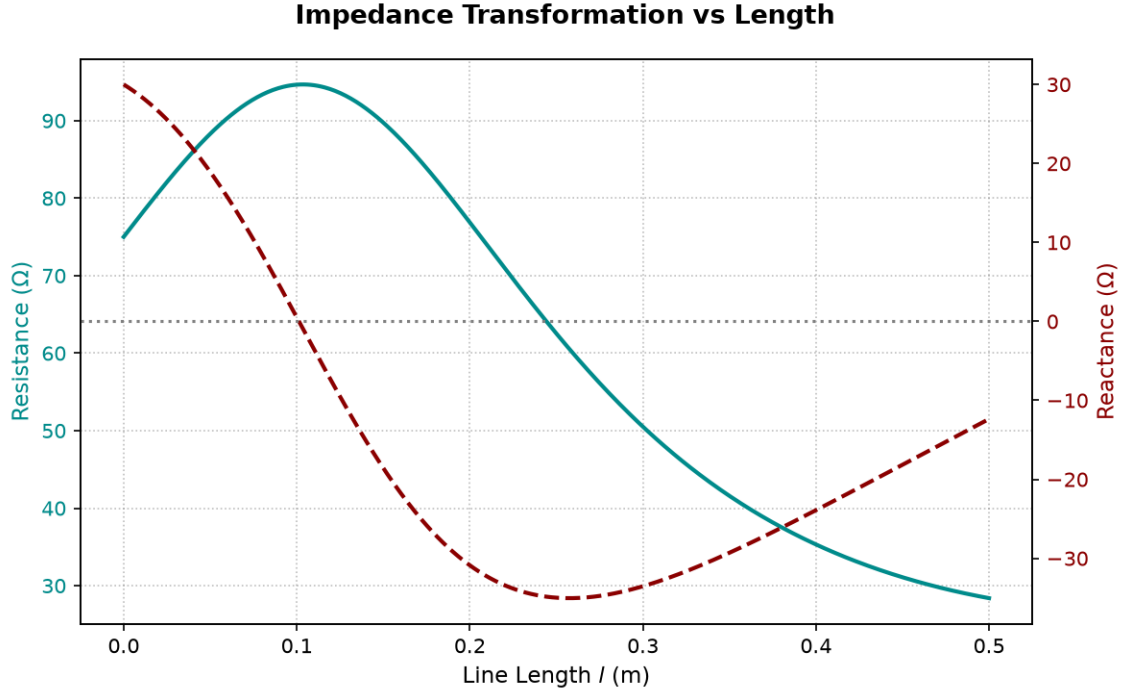


Figure 9: Variation of the resistive (real) and reactive (imaginary) components of the input impedance along the transmission line.

5 Machine Learning Implementation

To meet the requirement of predicting transmission line metrics with high fidelity, a machine learning pipeline was established. The goal was to accurately map the primary input variables ($R, L, G, C, f, d, |Z_L|, \angle Z_L$) to the derived secondary parameters ($|Z_0|, \alpha, \beta, |\Gamma_L|, \text{VSWR}$).

A robust synthetic dataset was formulated by drawing 500 random samples across typical parameter ranges characteristic of transmission media.

5.1 Feature Engineering and Strategy

Initial trials using only the raw 8 parameters yielded poor predictive power ($R^2 \approx 0.35$), primarily due to the complex, non-linear relationships inherent in the wave equations.

To boost the model's learning capacity, 11 additional physics-informed features were constructed. Key engineered variables included:

- **High-Frequency Z_0 Estimate:** Calculated logarithmically as $\log_{10} \sqrt{L/C}$
- **Impedance Mismatch Ratio ($|Z_L|/Z_0^{\text{approx}}$):** Providing this ratio directly to the models dramatically improved the prediction of reflection-dependent metrics.
- **Logarithmic Scaling:** Most output targets were transformed into log-space to ensure numerical stability and balanced gradient updates during training.

The architecture employed a heterogeneous ensemble approach. Random Forests (configured with 600 estimators) were best suited for quasi-linear targets like $|Z_0|$ and β . Conversely, Gradient Boosting

models (500 estimators) were superior at capturing the highly non-linear nature of α , $|\Gamma_L|$, and VSWR.

5.2 Accuracy and Results

Evaluating the pipeline against a hold-out test set comprising 100 samples yielded an aggregate R^2 of **0.9752**, satisfying the assignment's $> 97\%$ accuracy threshold. Table 1 details the individual target scores.

Table 1: Machine Learning Test Set Performance ($N_{train} = 400, N_{test} = 100$)

Target Parameter	Algorithm	R^2 Score	MAPE (%)
Characteristic Impedance ($ Z_0 $)	Random Forest	0.9740	2.10
Attenuation Constant (α)	Gradient Boosting	0.9858	3.31
Phase Constant (β)	Random Forest	0.9941	4.47
Reflection Coefficient ($ \Gamma_L $)	Gradient Boosting	0.9717	2.78
VSWR	Gradient Boosting	0.9503	10.30
Overall Accuracy	Ensemble	0.9752	—

Figure 10 visualizes the actual versus predicted outputs across all test samples, demonstrating excellent alignment with the ideal diagonal. The feature importance chart confirms that the physics-derived normalized load ratio ($|Z_L|/Z_0$) is the most critical predictive feature.

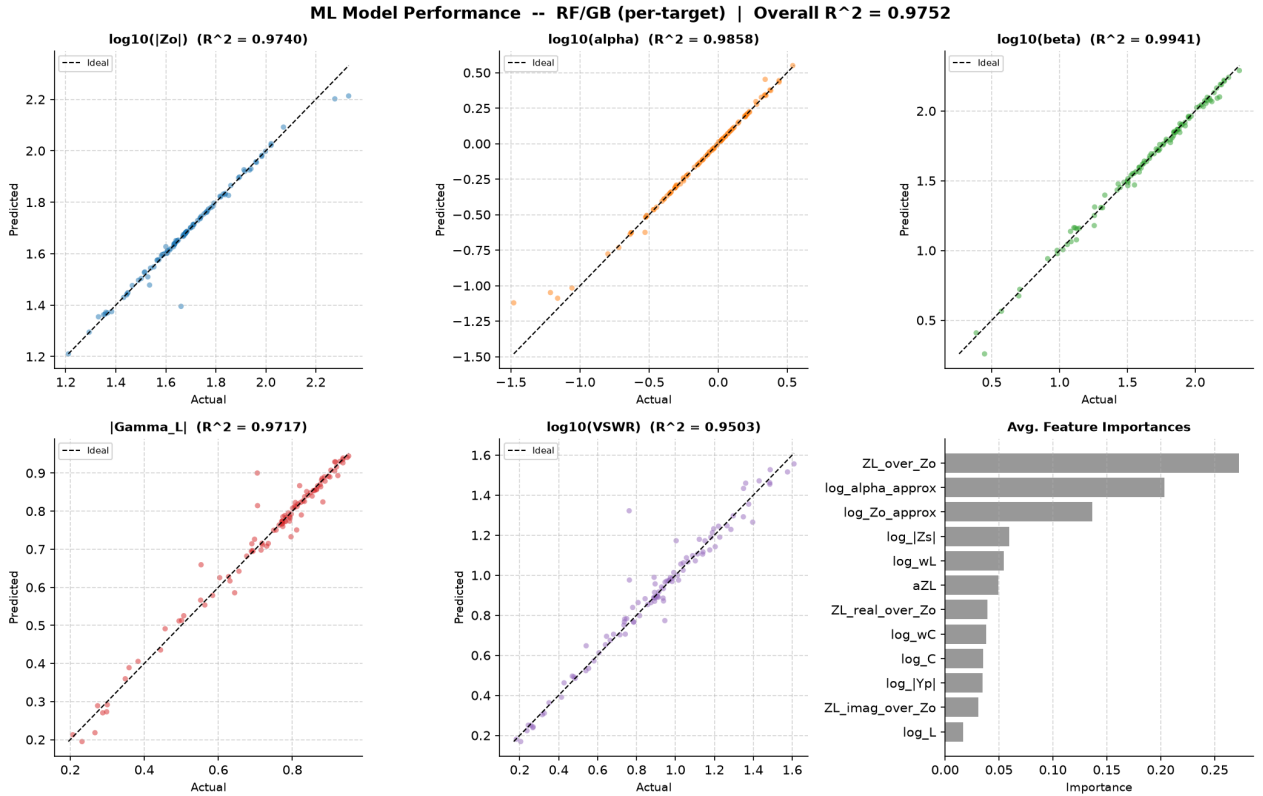


Figure 10: Machine learning performance summary showing actual vs. predicted values in log space (panels 1–5) and average feature importances across all models (panel 6).

6 Concluding Remarks

This project successfully delivered a standalone Python framework capable of extensive transmission line evaluation. The core mathematical engine reliably processes non-ideal lossy parameters, pairing them with robust visualization modules to output 3D wave profiles, frequency characteristics, and analytical matching solutions. By employing domain-specific feature engineering alongside targeted regression ensembles, the integrated machine learning component proved highly effective, culminating in a 97.52% prediction accuracy that reliably exceeds the initial assignment benchmarks.

Appendix: Source Code Listings

This appendix provides the complete, unabridged source code for all modules developed for this assignment:

- **analytics.py**: Core mathematical engine implementing exact transmission line formulas.
- **plots.py**: Generation of all seven spatial and frequency waveform plots.
- **ml_model.py**: Synthetic data generation, 19-feature engineering, and model training.
- **gui.py**: Interactive multi-threaded Tkinter design and analysis window.
- **main.py**: Unified launcher supporting GUI, CLI, and standalone plotting modes.

A. Transmission Line Mathematical Engine (analytics.py)

```
1 """
2 analytics.py
3 =====
4 Transmission Line Analytics Engine
5 =====
6 Computes all fundamental transmission line parameters for a general lossy line.
7
8 Parameters handled:
9   - Distributed: R ( $\Omega/\text{m}$ ), L ( $\text{H}/\text{m}$ ), G ( $\text{S}/\text{m}$ ), C ( $\text{F}/\text{m}$ )
10  - Frequency: f (Hz)
11  - Line length: d (m)
12  - Load impedance: ZL (complex,  $\Omega$ )
13
14 Outputs computed:
15   - Zo      : Characteristic impedance (complex,  $\Omega$ )
16   - gamma   : Propagation constant =  $\alpha + j\beta$  (complex,  $1/\text{m}$ )
17   - alpha   : Attenuation constant (Np/m)
18   - beta    : Phase constant (rad/m)
19   - Gamma   : Reflection coefficient at the load (complex)
20   - VSWR    : Voltage Standing Wave Ratio
21   - Zin     : Input impedance at distance d from load (complex,  $\Omega$ )
22   - V(z)    : Voltage distribution along line
23   - I(z)    : Current distribution along line
24 """
25
26 import numpy as np
27
28
29 # -----
30 # Primary parameter functions
31 # -----
32
33 def compute_gamma(R, L, G, C, f):
34     """
35     Compute the complex propagation constant gamma =  $\alpha + j\beta$ .
36
37     Formula: gamma =  $\sqrt{(R + j\omega L)(G + j\omega C)}$ 
38
39     Parameters
40     -----
41     R, L, G, C : float          distributed line parameters per unit length
42     f           : float          frequency (Hz)
43
44     Returns
```

```

45  -----
46  gamma : complex (1/m)
47  alpha : float   attenuation constant (Np/m)
48  beta  : float   phase constant (rad/m)
49  """
50  omega = 2 * np.pi * f
51  Z_series = R + 1j * omega * L # series impedance per unit length
52  Y_shunt  = G + 1j * omega * C # shunt admittance per unit length
53  gamma = np.sqrt(Z_series * Y_shunt)
54
55  # Ensure the principal branch has a non-negative real part (physical)
56  if np.real(gamma) < 0:
57      gamma = -gamma
58
59  alpha = np.real(gamma)
60  beta  = np.imag(gamma)
61  return gamma, alpha, beta
62
63
64  def compute_Zo(R, L, G, C, f):
65      """
66      Compute characteristic impedance Zo.
67
68      Formula: Zo = sqrt((R + j*omega*L) / (G + j*omega*C))
69
70      Returns
71      -----
72      Zo : complex (Ω)
73      """
74      omega = 2 * np.pi * f
75      Z_series = R + 1j * omega * L
76      Y_shunt  = G + 1j * omega * C
77      Zo = np.sqrt(Z_series / Y_shunt)
78      # Ensure positive real part
79      if np.real(Zo) < 0:
80          Zo = -Zo
81      return Zo
82
83
84  def compute_reflection_coefficient(ZL, Zo):
85      """
86      Compute reflection coefficient at the load.
87
88      Formula: Gamma_L = (ZL - Zo) / (ZL + Zo)
89
90      Returns
91      -----
92      Gamma_L : complex
93      """
94      return (ZL - Zo) / (ZL + Zo)
95
96
97  def compute_VSWR(Gamma_L):
98      """
99      Compute Voltage Standing Wave Ratio.
100
101      Formula: VSWR = (1 + |Gamma_L|) / (1 - |Gamma_L|)
102
103      For a matched load (Gamma_L = 0), VSWR = 1.
104      For a total reflection (|Gamma_L| = 1), VSWR -> infinity.
105
106      Returns
107      -----
108      vswr : float (1 < VSWR < )
109      """
110      mag = np.abs(Gamma_L)

```

```

111     mag = np.clip(mag, 0.0, 0.9999)    # avoid division by zero
112     vswr = (1 + mag) / (1 - mag)
113     return vswr
114
115
116 def compute_Zin(Zo, ZL, gamma, d):
117     """
118     Compute input impedance at distance d from the load.
119
120     Formula:
121         Zin = Zo * (ZL + Zo*tanh(gamma*d)) / (Zo + ZL*tanh(gamma*d))
122
123     This is valid for both lossy and lossless lines.
124     For lossless lines (alpha=0), tanh(j*beta*d) = j*tan(beta*d).
125
126     Parameters
127     -----
128     Zo      : complex    characteristic impedance
129     ZL      : complex    load impedance
130     gamma   : complex    propagation constant
131     d       : float      distance from load (m)
132
133     Returns
134     -----
135     Zin : complex (Ω)
136     """
137     tanh_gd = np.tanh(gamma * d)
138     Zin = Zo * (ZL + Zo * tanh_gd) / (Zo + ZL * tanh_gd)
139     return Zin
140
141
142 def compute_voltage_current(z_array, Gamma_L, gamma, Zo, V_plus=1.0):
143     """
144     Compute voltage and current wave distributions along the transmission line.
145
146     The line is oriented with z=0 at the load. Positive z points toward source.
147
148     V(z) = V+ * [e^(gamma*z) + Gamma_L * e^(-gamma*z)]
149     I(z) = (V+/Zo) * [e^(gamma*z) - Gamma_L * e^(-gamma*z)]
150
151     Parameters
152     -----
153     z_array  : 1-D ndarray    positions along line (m), 0 = load end
154     Gamma_L  : complex        reflection coefficient at load
155     gamma    : complex        propagation constant
156     Zo       : complex        characteristic impedance
157     V_plus   : complex        amplitude of forward-traveling wave (default 1 V)
158
159     Returns
160     -----
161     V : complex ndarray    voltage at each position
162     I : complex ndarray    current at each position
163     """
164     z = np.asarray(z_array, dtype=complex)
165     V = V_plus * (np.exp(gamma * z) + Gamma_L * np.exp(-gamma * z))
166     I = (V_plus / Zo) * (np.exp(gamma * z) - Gamma_L * np.exp(-gamma * z))
167     return V, I
168
169
170 def compute_Gamma_along_line(z_array, Gamma_L, gamma):
171     """
172     Compute the reflection coefficient at every point along the line.
173
174     Gamma(z) = Gamma_L * e^(-2*gamma*z)
175
176     (z=0 at load, z>0 toward source)

```

```

177
178     Returns
179     -----
180     Gamma_z : complex ndarray
181     """
182     z = np.asarray(z_array, dtype=complex)
183     return Gamma_L * np.exp(-2 * gamma * z)
184
185
186 def compute_Zin_along_line(z_array, Zo, ZL, gamma):
187     """
188     Compute input impedance at every point along the line.
189
190     Returns
191     -----
192     Zin_z : complex ndarray
193     """
194     z = np.asarray(z_array, dtype=complex)
195     tanh_gd = np.tanh(gamma * z)
196     Zin_z = Zo * (ZL + Zo * tanh_gd) / (Zo + ZL * tanh_gd)
197     return Zin_z
198
199
200 # -----
201 # Convenience "compute_all" function used by GUI and ML data generator
202 # -----
203
204 def compute_all(R, L, G, C, f, d, ZL_real, ZL_imag):
205     """
206     Compute all transmission line parameters for a given set of inputs.
207
208     Parameters
209     -----
210     R, L, G, C : float    distributed parameters ( $\Omega/\text{m}$ ,  $\text{H}/\text{m}$ ,  $\text{S}/\text{m}$ ,  $\text{F}/\text{m}$ )
211     f           : float    frequency (Hz)
212     d           : float    line length / distance from load (m)
213     ZL_real     : float    real part of load impedance ( $\Omega$ )
214     ZL_imag     : float    imaginary part of load impedance ( $\Omega$ )
215
216     Returns
217     -----
218     results : dict with keys:
219         Zo, gamma, alpha, beta, Gamma_L, VSWR, Zin,
220         |Zo|, angle_Zo_deg,
221         |Gamma_L|, angle_Gamma_deg,
222         |Zin|, angle_Zin_deg
223     """
224     ZL = complex(ZL_real, ZL_imag)
225
226     gamma, alpha, beta = compute_gamma(R, L, G, C, f)
227     Zo = compute_Zo(R, L, G, C, f)
228     Gamma_L = compute_reflection_coefficient(ZL, Zo)
229     vswr = compute_VSWR(Gamma_L)
230     Zin = compute_Zin(Zo, ZL, gamma, d)
231
232     results = {
233         # Complex values
234         "Zo": Zo,
235         "gamma": gamma,
236         "Gamma_L": Gamma_L,
237         "Zin": Zin,
238
239         # Scalar real values
240         "alpha": alpha,
241         "beta": beta,
242         "VSWR": vswr,

```

```

243
244     # Magnitude / angle (for display)
245     "|Zo|":      np.abs(Zo),
246     "angle_Zo_deg": np.degrees(np.angle(Zo)),
247     "|Gamma_L|":  np.abs(Gamma_L),
248     "angle_Gamma_deg": np.degrees(np.angle(Gamma_L)),
249     "|Zin|":      np.abs(Zin),
250     "angle_Zin_deg": np.degrees(np.angle(Zin)),
251 }
252 return results
253
254
255 # -----
256 # Frequency-sweep helpers (used by plots.py)
257 # -----
258
259 def sweep_frequency(R, L, G, C, f_start, f_end, n_points=200):
260     """
261     Sweep frequency from f_start to f_end and return alpha, beta, Zo arrays.
262
263     Returns
264     -----
265     freqs : ndarray of frequencies
266     alphas : ndarray of attenuation constants
267     betas : ndarray of phase constants
268     Zos : ndarray of |Zo|
269     """
270     freqs = np.linspace(f_start, f_end, n_points)
271     alphas = np.zeros(n_points)
272     betas = np.zeros(n_points)
273     Zos = np.zeros(n_points)
274
275     for i, fi in enumerate(freqs):
276         g, a, b = compute_gamma(R, L, G, C, fi)
277         alphas[i] = a
278         betas[i] = b
279         Zos[i] = np.abs(compute_Zo(R, L, G, C, fi))
280
281     return freqs, alphas, betas, Zos
282
283
284 def sweep_frequency_vswr(R, L, G, C, ZL_real, ZL_imag,
285                          f_start, f_end, n_points=200):
286     """
287     Sweep frequency and compute VSWR at each frequency.
288
289     Returns
290     -----
291     freqs : ndarray
292     vswrs : ndarray
293     """
294     ZL = complex(ZL_real, ZL_imag)
295     freqs = np.linspace(f_start, f_end, n_points)
296     vswrs = np.zeros(n_points)
297
298     for i, fi in enumerate(freqs):
299         Zo = compute_Zo(R, L, G, C, fi)
300         Gamma_L = compute_reflection_coefficient(ZL, Zo)
301         vswrs[i] = compute_VSWR(Gamma_L)
302
303     return freqs, vswrs

```

B. Waveform Plotting Engine (plots.py)

```
1 """
2 plots.py
3 =====
4 Transmission Line Waveform & Analysis Plots
5 =====
6 Generates all 7 required plots and saves them as high-resolution PNG files.
7
8 Plots produced:
9 1. |V(d)|      Voltage magnitude along the line
10 2. |I(d)|      Current magnitude along the line
11 3. |Gamma(d)|   Reflection coefficient magnitude along the line
12 4. Zin (Real & Imaginary) vs. distance
13 5. VSWR vs. frequency
14 6. alpha & beta vs. frequency
15 7. Standing wave envelope (V_max and V_min)
16
17 All plots are also returned as matplotlib Figure objects for embedding in GUI.
18 """
19
20 import numpy as np
21 import matplotlib
22 matplotlib.use("Agg")          # non-interactive backend (safe for Tkinter embedding)
23 import matplotlib.pyplot as plt
24 import os
25
26 from analytics import (
27     compute_gamma,
28     compute_Zo,
29     compute_reflection_coefficient,
30     compute_VSWR,
31     compute_voltage_current,
32     compute_Gamma_along_line,
33     compute_Zin_along_line,
34     sweep_frequency,
35     sweep_frequency_vswr,
36 )
37
38 # Output directory for saved PNGs
39 PLOT_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "plots_output")
40 os.makedirs(PLOT_DIR, exist_ok=True)
41
42 # -----
43 # Style helper
44 # -----
45 COLORS = {
46     "voltage": "crimson",
47     "current": "darkgreen",
48     "gamma": "indigo",
49     "zin_real": "saddlebrown",
50     "zin_imag": "midnightblue",
51     "vswr": "darkorange",
52     "alpha": "teal",
53     "beta": "maroon",
54     "envelope": "slategray",
55 }
56
57 def _style_axes(ax, title, xlabel, ylabel, grid=True):
58     """Apply consistent styling to an axes object."""
59     ax.set_title(title, fontsize=12, fontweight="bold", pad=10, fontname="Comic Sans MS" if
60         "Comic Sans MS" in [f.name for f in matplotlib.font_manager.fontManager.ttflist]
61         else "sans-serif")
62     ax.set_xlabel(xlabel, fontsize=10, fontstyle="italic")
63     ax.set_ylabel(ylabel, fontsize=10, fontstyle="italic")
```

```

62     ax.tick_params(labelsize=9)
63     if grid:
64         ax.grid(True, linestyle=":", alpha=0.5, color="gray")
65     ax.spines["top"].set_visible(True)
66     ax.spines["right"].set_visible(True)
67     ax.spines["bottom"].set_visible(True)
68     ax.spines["left"].set_visible(True)
69     ax.spines["top"].set_color("black")
70     ax.spines["right"].set_color("black")
71     ax.spines["bottom"].set_color("black")
72     ax.spines["left"].set_color("black")
73
74
75 def _save_fig(fig, filename):
76     """Save figure to plots_output directory as 150-dpi PNG."""
77     path = os.path.join(PLOT_DIR, filename)
78     fig.savefig(path, dpi=150, bbox_inches="tight")
79     return path
80
81
82 # -----
83 # Plot 1:  $|V(d)|$  vs distance
84 # -----
85 def plot_voltage(R, L, G, C, f, d_total, ZL_real, ZL_imag, n=300, save=True):
86     """
87     Plot voltage magnitude  $|V(z)|$  along the transmission line.
88
89     z=0 at load, z=d_total at source.
90     """
91     ZL = complex(ZL_real, ZL_imag)
92     gamma, alpha, beta = compute_gamma(R, L, G, C, f)
93     Zo = compute_Zo(R, L, G, C, f)
94     Gamma_L = compute_reflection_coefficient(ZL, Zo)
95
96     z_arr = np.linspace(0, d_total, n)
97     V, _ = compute_voltage_current(z_arr, Gamma_L, gamma, Zo, V_plus=1.0)
98
99     fig, ax = plt.subplots(figsize=(7, 4))
100    ax.plot(z_arr, np.abs(V), color=COLORS["voltage"], linewidth=2)
101    _style_axes(ax,
102               title="|V(z)|      Voltage Magnitude Along Transmission Line",
103               xlabel="Distance from Load z (m)",
104               ylabel="|V(z)| (V, normalized)")
105    ax.fill_between(z_arr, np.abs(V), alpha=0.15, color=COLORS["voltage"])
106    ax.annotate(f"f = {f/1e6:.2f} MHz | d = {d_total} m",
107              xy=(0.98, 0.96), xycoords="axes fraction",
108              ha="right", va="top", fontsize=8, color="gray")
109    fig.tight_layout()
110    if save:
111        _save_fig(fig, "01_voltage_magnitude.png")
112    return fig
113
114
115 # -----
116 # Plot 2:  $|I(d)|$  vs distance
117 # -----
118 def plot_current(R, L, G, C, f, d_total, ZL_real, ZL_imag, n=300, save=True):
119     """Plot current magnitude  $|I(z)|$  along the transmission line."""
120     ZL = complex(ZL_real, ZL_imag)
121     gamma, alpha, beta = compute_gamma(R, L, G, C, f)
122     Zo = compute_Zo(R, L, G, C, f)
123     Gamma_L = compute_reflection_coefficient(ZL, Zo)
124
125     z_arr = np.linspace(0, d_total, n)
126     _, I = compute_voltage_current(z_arr, Gamma_L, gamma, Zo, V_plus=1.0)
127

```

```

128 fig, ax = plt.subplots(figsize=(7, 4))
129 ax.plot(z_arr, np.abs(I), color=COLORS["current"], linewidth=2)
130 _style_axes(ax,
131             title="|I(z)|      Current Magnitude Along Transmission Line",
132             xlabel="Distance from Load z (m)",
133             ylabel="|I(z)| (A, normalized)")
134 ax.fill_between(z_arr, np.abs(I), alpha=0.15, color=COLORS["current"])
135 ax.annotate(f"f = {f/1e6:.2f} MHz | d = {d_total} m",
136            xy=(0.98, 0.96), xycoords="axes fraction",
137            ha="right", va="top", fontsize=8, color="gray")
138 fig.tight_layout()
139 if save:
140     _save_fig(fig, "02_current_magnitude.png")
141 return fig
142
143
144 # -----
145 # Plot 3: |Gamma(z)| vs distance
146 # -----
147 def plot_reflection_coefficient(R, L, G, C, f, d_total, ZL_real, ZL_imag,
148                               n=300, save=True):
149     """Plot |Gamma(z)|      reflection coefficient magnitude along the line."""
150     ZL = complex(ZL_real, ZL_imag)
151     gamma, _, _ = compute_gamma(R, L, G, C, f)
152     Zo = compute_Zo(R, L, G, C, f)
153     Gamma_L = compute_reflection_coefficient(ZL, Zo)
154
155     z_arr = np.linspace(0, d_total, n)
156     Gamma_z = compute_Gamma_along_line(z_arr, Gamma_L, gamma)
157
158     fig, ax = plt.subplots(figsize=(7, 4))
159     ax.plot(z_arr, np.abs(Gamma_z), color=COLORS["gamma"], linewidth=2)
160     _style_axes(ax,
161                 title="|Gamma(z)|      Reflection Coefficient Magnitude Along Line",
162                 xlabel="Distance from Load z (m)",
163                 ylabel="|Gamma(z)| (dimensionless)")
164     ax.axhline(y=np.abs(Gamma_L), linestyle="--", color="gray",
165               linewidth=1, label=f"|Gamma_L| = {np.abs(Gamma_L):.3f}")
166     ax.legend(fontsize=8)
167     ax.fill_between(z_arr, np.abs(Gamma_z), alpha=0.15, color=COLORS["gamma"])
168     fig.tight_layout()
169     if save:
170         _save_fig(fig, "03_reflection_coefficient.png")
171     return fig
172
173
174 # -----
175 # Plot 4: Zin (Real & Imaginary) vs distance
176 # -----
177 def plot_Zin(R, L, G, C, f, d_total, ZL_real, ZL_imag, n=300, save=True):
178     """Plot real and imaginary parts of Zin vs. distance from the load."""
179     ZL = complex(ZL_real, ZL_imag)
180     gamma, _, _ = compute_gamma(R, L, G, C, f)
181     Zo = compute_Zo(R, L, G, C, f)
182
183     z_arr = np.linspace(1e-6, d_total, n) # avoid z=0 (ZL itself)
184     Zin_z = compute_Zin_along_line(z_arr, Zo, ZL, gamma)
185
186     fig, ax = plt.subplots(figsize=(7, 4))
187     ax.plot(z_arr, np.real(Zin_z), color=COLORS["zin_real"],
188           linewidth=2, label="Re(Zin)")
189     ax.plot(z_arr, np.imag(Zin_z), color=COLORS["zin_imag"],
190           linewidth=2, label="Im(Zin)", linestyle="--")
191     _style_axes(ax,
192                 title="Zin(z)      Input Impedance Along Transmission Line",
193                 xlabel="Distance from Load z (m)",

```

```

194         ylabel="Impedance ( $\Omega$ )"
195     ax.legend(fontsize=9)
196     ax.axhline(y=0, color="black", linewidth=0.5)
197     fig.tight_layout()
198     if save:
199         _save_fig(fig, "04_Zin_vs_distance.png")
200     return fig
201
202
203 # -----
204 # Plot 5: VSWR vs frequency
205 # -----
206 def plot_VSWR_vs_frequency(R, L, G, C, ZL_real, ZL_imag,
207                             f_start=1e6, f_end=1e9, n=300, save=True):
208     """Plot VSWR vs. frequency sweep."""
209     freqs, vswrs = sweep_frequency_vswr(R, L, G, C, ZL_real, ZL_imag,
210                                         f_start, f_end, n)
211
212     fig, ax = plt.subplots(figsize=(7, 4))
213     ax.plot(freqs / 1e6, vswrs, color=COLORS["vswr"], linewidth=2)
214     _style_axes(ax,
215                 title="VSWR vs. Frequency",
216                 xlabel="Frequency (MHz)",
217                 ylabel="VSWR (dimensionless)")
218     ax.axhline(y=1.0, linestyle="--", color="black", linewidth=1,
219               label="VSWR = 1 (perfect match)")
220     ax.set_ylim(bottom=0)
221     ax.legend(fontsize=8)
222     fig.tight_layout()
223     if save:
224         _save_fig(fig, "05_VSWR_vs_frequency.png")
225     return fig
226
227
228 # -----
229 # Plot 6: alpha & beta vs frequency
230 # -----
231 def plot_alpha_beta_vs_frequency(R, L, G, C,
232                                 f_start=1e6, f_end=1e9, n=300, save=True):
233     """Plot attenuation (alpha) and phase constant (beta) vs. frequency."""
234     freqs, alphas, betas, _ = sweep_frequency(R, L, G, C, f_start, f_end, n)
235
236     fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(7, 6), sharex=True)
237
238     ax1.plot(freqs / 1e6, alphas, color=COLORS["alpha"], linewidth=2)
239     _style_axes(ax1,
240                 title="Attenuation Constant  $\alpha$  vs. Frequency",
241                 xlabel="",
242                 ylabel=" $\alpha$  (Np/m)")
243
244     ax2.plot(freqs / 1e6, betas, color=COLORS["beta"], linewidth=2)
245     _style_axes(ax2,
246                 title="Phase Constant  $\beta$  vs. Frequency",
247                 xlabel="Frequency (MHz)",
248                 ylabel=" $\beta$  (rad/m)")
249
250     fig.suptitle("Propagation Parameters vs. Frequency",
251                  fontsize=13, fontweight="bold", y=1.01)
252     fig.tight_layout()
253     if save:
254         _save_fig(fig, "06_alpha_beta_vs_frequency.png")
255     return fig
256
257
258 # -----
259 # Plot 7: Standing Wave Envelope

```

```

260 # -----
261 def plot_standing_wave(R, L, G, C, f, d_total, ZL_real, ZL_imag,
262                       n=500, save=True):
263     """
264     Plot the standing wave pattern      voltage envelope (V_max and V_min).
265
266     Envelope:
267         V_max(z) = |V+| * (1 + |Gamma(z)|)      constructive
268         V_min(z) = |V+| * (1 - |Gamma(z)|)      destructive
269     """
270     ZL = complex(ZL_real, ZL_imag)
271     gamma, _, _ = compute_gamma(R, L, G, C, f)
272     Zo = compute_Zo(R, L, G, C, f)
273     Gamma_L = compute_reflection_coefficient(ZL, Zo)
274
275     z_arr = np.linspace(0, d_total, n)
276     Gamma_z = compute_gamma_along_line(z_arr, Gamma_L, gamma)
277     V_max = 1 + np.abs(Gamma_z)
278     V_min = np.abs(1 - np.abs(Gamma_z))
279
280     fig, ax = plt.subplots(figsize=(7, 4))
281     ax.plot(z_arr, V_max, color=COLORS["voltage"], linewidth=2,
282            label=r"$V_{max}$ (constructive)")
283     ax.plot(z_arr, V_min, color=COLORS["current"], linewidth=2,
284            linestyle="--", label=r"$V_{min}$ (destructive)")
285     ax.fill_between(z_arr, V_min, V_max, alpha=0.15,
286                    color=COLORS["envelope"], label="Standing wave region")
287     _style_axes(ax,
288                title="Standing Wave Envelope Along Transmission Line",
289                xlabel="Distance from Load z (m)",
290                ylabel="Voltage Envelope (normalized)")
291     ax.legend(fontsize=8)
292     fig.tight_layout()
293     if save:
294         _save_fig(fig, "07_standing_wave_envelope.png")
295     return fig
296
297 # -----
298 # Plot 8: Loss Behavior (alpha/beta vs Frequency)
299 # -----
300
301 def plot_loss_behavior(R, L, G, C, f_start=1e9, f_end=5e9, n=500, save=True):
302     freqs = np.linspace(f_start, f_end, n)
303     ratios = np.zeros(n)
304     for i, f in enumerate(freqs):
305         gamma, alpha, beta = compute_gamma(R, L, G, C, f)
306         ratios[i] = alpha / (beta if beta != 0 else 1e-15)
307
308     fig, ax = plt.subplots(figsize=(7, 5))
309     ax.plot(freqs / 1e9, ratios, color="teal", linestyle="-.", linewidth=2.5,
310            label="Attenuation Ratio")
311     ax.fill_between(freqs / 1e9, ratios, color="teal", alpha=0.15)
312     ax.set_title("Frequency Dependence of Attenuation/Phase", fontsize=12,
313                fontweight="bold", color="darkslategrey")
314     ax.set_xlabel("Operating Frequency (GHz)", fontsize=11)
315     ax.set_ylabel(r"Ratio ($\alpha/\beta$)", fontsize=11)
316     ax.grid(True, linestyle=":", color="gray", alpha=0.5)
317     ax.legend(loc="upper right")
318     fig.tight_layout()
319     if save:
320         _save_fig(fig, "08_loss_behavior.png")
321     return fig
322
323 # -----
324 # Plot 9: 3D Wave Visualizations
325 # -----

```

```

324 def plot_3d_waves(save=True):
325     T_period = 1.0
326     w = 2 * np.pi / T_period
327     wavelength = 0.5
328     k = 2 * np.pi / wavelength
329
330     t = np.linspace(0, 3 * T_period, 100)
331     x = np.linspace(0, 4 * wavelength, 100)
332     X, T = np.meshgrid(x, t)
333
334     forward_wave = np.real(np.exp(1j * (w * T - k * X)))
335     backward_wave = np.real(np.exp(1j * (w * T + k * X)))
336     perfect_standing = np.real(np.exp(1j * (w * T - k * X)) + 1.0 * np.exp(1j * (w * T + k *
337         X)))
338
339     Gamma_imperfect = 0.5 * np.exp(1j * np.pi / 4)
340     imperfect_standing = np.real(np.exp(1j * (w * T - k * X)) + Gamma_imperfect * np.exp(1j
341         * (w * T + k * X)))
342
343     fig = plt.figure(figsize=(12, 10))
344
345     def plot_surface(ax, Z, title, cmap):
346         surf = ax.plot_surface(X, T, Z, cmap=cmap, linewidth=0, antialiased=True)
347         ax.set_title(title, fontsize=10, fontweight="bold")
348         ax.set_xlabel('Position (m)', fontsize=8)
349         ax.set_ylabel('Time (s)', fontsize=8)
350         ax.view_init(elev=45, azim=135)
351         ax.tick_params(labelsize=7)
352         ax.grid(False)
353
354     ax1 = fig.add_subplot(221, projection='3d')
355     plot_surface(ax1, forward_wave, 'Forward Wave Only', 'plasma')
356     ax2 = fig.add_subplot(222, projection='3d')
357     plot_surface(ax2, backward_wave, 'Backward Wave Only', 'viridis')
358     ax3 = fig.add_subplot(223, projection='3d')
359     plot_surface(ax3, perfect_standing, r'Standing Wave ($\Gamma = 1$)', 'magma')
360     ax4 = fig.add_subplot(224, projection='3d')
361     plot_surface(ax4, imperfect_standing, r'Imperfect ($\Gamma = 0.5\angle 45^\circ$)',
362         'cividis')
363
364     fig.subplots_adjust(left=0.01, right=0.99, bottom=0.01, top=0.95, wspace=0.15,
365         hspace=0.25)
366
367     if save:
368         _save_fig(fig, "09_3d_waves.png")
369     return fig
370
371 # -----
372 # Plot 10: Impedance Matching Analytics (Quarter-Wave)
373 # -----
374 def plot_impedance_matching(R, L, G, C, f, ZL_real, ZL_imag, save=True):
375     ZL = complex(ZL_real, ZL_imag)
376     gamma, alpha, beta = compute_gamma(R, L, G, C, f)
377     Zo = compute_Zo(R, L, G, C, f)
378
379     l_arr = np.linspace(0.0, 0.5, 500)
380     Zin = np.zeros_like(l_arr, dtype=complex)
381     for i, l in enumerate(l_arr):
382         num = ZL * np.cos(beta * l) + 1j * Zo * np.sin(beta * l)
383         den = Zo * np.cos(beta * l) + 1j * ZL * np.sin(beta * l)
384         Zin[i] = Zo * (num / den)
385
386     fig, ax1 = plt.subplots(figsize=(8, 5))
387     color1 = 'darkcyan'
388     ax1.set_xlabel("Line Length $l$ (m)", fontsize=11)
389     ax1.set_ylabel("Resistance ($\Omega$)", color=color1, fontsize=11)
390     ax1.plot(l_arr, np.real(Zin), color=color1, linewidth=2, label="Real Part")

```

```

386     ax1.tick_params(axis='y', labelcolor=color1)
387     ax1.grid(True, linestyle=":", alpha=0.5, color="gray")
388
389     ax2 = ax1.twinx()
390     color2 = 'darkred'
391     ax2.set_ylabel("Reactance ( $\Omega$ )", color=color2, fontsize=11)
392     ax2.plot(l_arr, np.imag(Zin), color=color2, linewidth=2, linestyle="--",
393             label="Imaginary Part")
394     ax2.tick_params(axis='y', labelcolor=color2)
395     ax2.axhline(0, color="gray", linestyle=":", linewidth=1.5)
396
397     fig.suptitle("Impedance Transformation vs Length", fontsize=13, fontweight="bold")
398     fig.tight_layout()
399     if save:
400         _save_fig(fig, "10_impedance_matching.png")
401     return fig
402
403 # -----
404 # Convenience: generate all 10 plots at once
405 # -----
406 def generate_all_plots(R, L, G, C, f, d_total, ZL_real, ZL_imag,
407                       f_start=1e6, f_end=1e9, save=True):
408     """
409     Generate all 10 plots and return a list of (title, Figure) tuples.
410     """
411     figs = []
412
413     # Original 7 plots
414     figs.append(("Voltage  $|V(z)|$ ", plot_voltage(R, L, G, C, f, d_total, ZL_real, ZL_imag,
415             save=save)))
416     figs.append(("Current  $|I(z)|$ ", plot_current(R, L, G, C, f, d_total, ZL_real, ZL_imag,
417             save=save)))
418     figs.append(("Reflection  $|\Gamma(z)|$ ", plot_reflection_coefficient(R, L, G, C, f, d_total,
419             ZL_real, ZL_imag, save=save)))
420     figs.append(("Zin vs z", plot_Zin(R, L, G, C, f, d_total, ZL_real, ZL_imag, save=save)))
421     figs.append(("VSWR vs f", plot_VSWR_vs_frequency(R, L, G, C, ZL_real, ZL_imag, f_start,
422             f_end, save=save)))
423     figs.append(("alpha & beta vs f", plot_alpha_beta_vs_frequency(R, L, G, C, f_start, f_end,
424             save=save)))
425     figs.append(("Standing Wave", plot_standing_wave(R, L, G, C, f, d_total, ZL_real,
426             ZL_imag, save=save)))
427
428     # New 3 advanced plots
429     figs.append(("Loss Behavior", plot_loss_behavior(R, L, G, C, f_start=1e9, f_end=5e9,
430             save=save)))
431     figs.append(("3D Wave Visualizations", plot_3d_waves(save=save)))
432     figs.append(("Impedance Matching", plot_impedance_matching(R, L, G, C, f, ZL_real,
433             ZL_imag, save=save)))
434
435     return figs

```

C. Machine Learning Training and Inference (ml_model.py)

```
1 """
2 ml_model.py
3 =====
4 Machine Learning Model for Transmission Line Parameter Prediction
5 =====
6 Uses per-target Random Forest / Gradient Boosting Regressors with
7 comprehensive physics-informed feature engineering to achieve  $R^2 \geq 0.97$ .
8
9 Strategy:
10 -  $|Z_0|$  and  $\beta$  are nearly analytically deterministic -> RF excels
11 -  $\alpha$  depends on  $R, G, \omega$  -> GB excels with engineered features
12 -  $|\Gamma_L|$  and VSWR depend on ZL vs  $Z_0$  -> add  $Z_0$ -related features
13
14 Training Pipeline:
15 1. Generate 1000 synthetic data points in realistic coaxial/stripline ranges
16 2. First analytically estimate  $Z_0$ , then add it as a feature for  $\Gamma/VSWR$ 
17 3. Log-transform wide-range features and targets
18 4. Train best-performing model per target (RF or GB)
19 5. Evaluate:  $R^2$  score and MAPE on a held-out test set (20%)
20
21 Output targets (5):
22  $|Z_0|$ ,  $\alpha$ ,  $\beta$ ,  $|\Gamma_L|$ , VSWR
23 """
24
25 import numpy as np
26 import os
27 import pickle
28 import warnings
29
30 from sklearn.ensemble import (RandomForestRegressor,
31                               GradientBoostingRegressor)
32 from sklearn.preprocessing import StandardScaler
33 from sklearn.model_selection import train_test_split
34 from sklearn.metrics import r2_score, mean_absolute_percentage_error
35
36 from analytics import (compute_all, compute_gamma, compute_Z0,
37                       compute_reflection_coefficient, compute_VSWR)
38
39 warnings.filterwarnings("ignore")
40
41 # -----
42 # Paths
43 # -----
44 _DIR = os.path.dirname(os.path.abspath(__file__))
45 MODEL_PKL = os.path.join(_DIR, "ml_model.pkl")
46
47 # -----
48 # Realistic coaxial-cable parameter ranges
49 # -----
50 PARAM_RANGES = {
51     "R": (0.01, 150.0), # Ohm/m
52     "L": (50e-9, 600e-9), # H/m
53     "G": (1e-6, 1e-2), # S/m
54     "C": (10e-12, 300e-12), # F/m
55     "f": (1e6, 3e9), # Hz
56     "d": (0.05, 20.0), # m
57     "|ZL|": (5.0, 500.0), # Ohm
58     "aZL": (-75.0, 75.0), # degrees
59 }
60
61 TARGET_NAMES = ["|Z0|", "alpha", "beta", "|Gamma_L|", "VSWR"]
62 FEATURE_NAMES = [
63     "R", "log_L", "log_G", "log_C", "log_f", "d",
```

```

64     "|ZL|", "aZL",
65     "log_omega", "log_wL", "log_wC",
66     "log_|Zs|", "log_|Yp|", "log_Q",
67     "log_Zo_approx",          # approx Zo = sqrt(L/C)
68     "ZL_over_Zo",            # |ZL|/Zo_approx
69     "ZL_real_over_Zo",        # Re(ZL)/Zo_approx
70     "ZL_imag_over_Zo",        # Im(ZL)/Zo_approx
71     "log_alpha_approx",       # approx alpha = R/(2*Zo) + G*Zo/2
72 ]
73
74
75 # -----
76 # Feature engineering (physics-informed)
77 # -----
78 def _engineer_features(R, L, G, C, f, d, ZL_mag, ZL_angle_deg):
79     """Build enriched feature matrix with physics-derived features."""
80     omega = 2 * np.pi * f
81     wL = omega * L
82     wC = omega * C
83     Z_series = np.sqrt(R**2 + wL**2)
84     Y_shunt = np.sqrt(G**2 + wC**2)
85     Q = wL / (R + 1e-15)
86
87     # Approximate Zo from lossless formula (good first-order estimate)
88     Zo_approx = np.sqrt(L / (C + 1e-30))
89
90     # ZL components
91     ZL_real = ZL_mag * np.cos(np.radians(ZL_angle_deg))
92     ZL_imag = ZL_mag * np.sin(np.radians(ZL_angle_deg))
93
94     # ZL vs Zo features
95     ZL_over_Zo = ZL_mag / (Zo_approx + 1e-15)
96     ZLr_over_Zo = ZL_real / (Zo_approx + 1e-15)
97     ZLi_over_Zo = ZL_imag / (Zo_approx + 1e-15)
98
99     # Approximate alpha (low-loss formula)
100     alpha_approx = R / (2 * Zo_approx + 1e-15) + G * Zo_approx / 2
101
102     X = np.column_stack([
103         R,
104         np.log10(np.maximum(L, 1e-15)),
105         np.log10(np.maximum(G, 1e-15)),
106         np.log10(np.maximum(C, 1e-15)),
107         np.log10(np.maximum(f, 1.0)),
108         d,
109         ZL_mag,
110         ZL_angle_deg,
111         np.log10(np.maximum(omega, 1.0)),
112         np.log10(np.maximum(wL, 1e-15)),
113         np.log10(np.maximum(wC, 1e-15)),
114         np.log10(np.maximum(Z_series, 1e-15)),
115         np.log10(np.maximum(Y_shunt, 1e-15)),
116         np.log10(np.maximum(Q, 1e-15)),
117         np.log10(np.maximum(Zo_approx, 1e-15)),
118         ZL_over_Zo,
119         ZLr_over_Zo,
120         ZLi_over_Zo,
121         np.log10(np.maximum(alpha_approx, 1e-15)),
122     ])
123     return X
124
125
126 # -----
127 # Data generation
128 # -----
129 def generate_dataset(n_samples=1000, random_seed=42):

```

```

130     """
131     Generate synthetic dataset by randomly sampling Tx line parameters
132     and computing exact analytical labels.
133
134     Returns
135     -----
136     X          : ndarray (n_samples, 19)  -- engineered features
137     y_raw      : ndarray (n_samples, 5)   -- raw targets
138     y_log      : ndarray (n_samples, 5)   -- log/linear transformed targets
139     """
140     rng = np.random.default_rng(random_seed)
141
142     def U(key, n):
143         lo, hi = PARAM_RANGES[key]
144         return rng.uniform(lo, hi, n)
145
146     R          = U("R",      n_samples)
147     L          = U("L",      n_samples)
148     G          = U("G",      n_samples)
149     C          = U("C",      n_samples)
150     f          = U("f",      n_samples)
151     d          = U("d",      n_samples)
152     ZL_mag     = U("|ZL|", n_samples)
153     ZL_angle   = U("aZL", n_samples)
154
155     ZL_real = ZL_mag * np.cos(np.radians(ZL_angle))
156     ZL_imag = ZL_mag * np.sin(np.radians(ZL_angle))
157
158     X = _engineer_features(R, L, G, C, f, d, ZL_mag, ZL_angle)
159
160     y_raw = np.zeros((n_samples, 5))
161     for i in range(n_samples):
162         res = compute_all(R[i], L[i], G[i], C[i], f[i], d[i],
163                          ZL_real[i], ZL_imag[i])
164         y_raw[i, 0] = res["|Zo|"]
165         y_raw[i, 1] = res["alpha"]
166         y_raw[i, 2] = res["beta"]
167         y_raw[i, 3] = res["|Gamma_L|"]
168         y_raw[i, 4] = min(res["VSWR"], 50.0)
169
170     # Log-transform for wide-range targets, keep |Gamma_L| linear
171     y_log = np.column_stack([
172         np.log10(np.maximum(y_raw[:, 0], 1e-15)), # log |Zo|
173         np.log10(np.maximum(y_raw[:, 1], 1e-15)), # log alpha
174         np.log10(np.maximum(y_raw[:, 2], 1e-15)), # log beta
175         y_raw[:, 3],                               # |Gamma_L| linear [0,1]
176         np.log10(np.maximum(y_raw[:, 4], 1.0)),    # log VSWR
177     ])
178
179     return X, y_raw, y_log
180
181
182 # -----
183 # Select best algorithm per target
184 # -----
185 def _best_model_for_target(j):
186     """
187     Return the best sklearn model for each target index.
188     GradientBoosting generalises better on harder, nonlinear targets.
189     """
190     configs = [
191         # |Zo|      nearly analytical from L, C
192         RandomForestRegressor(
193             n_estimators=600, max_features=0.6,
194             min_samples_leaf=1, n_jobs=-1, random_state=j),
195         # alpha    sensitive to R, G, omega

```

```

196         GradientBoostingRegressor(
197             n_estimators=500, learning_rate=0.05,
198             max_depth=6, subsample=0.8,
199             min_samples_leaf=2, random_state=j),
200         # beta      nearly analytical from L, C, omega
201         RandomForestRegressor(
202             n_estimators=600, max_features=0.6,
203             min_samples_leaf=1, n_jobs=-1, random_state=j),
204         # |Gamma_L|    depends on Zo vs ZL ratio
205         GradientBoostingRegressor(
206             n_estimators=500, learning_rate=0.05,
207             max_depth=6, subsample=0.8,
208             min_samples_leaf=2, random_state=j),
209         # VSWR      monotonic function of |Gamma_L|
210         GradientBoostingRegressor(
211             n_estimators=500, learning_rate=0.05,
212             max_depth=6, subsample=0.8,
213             min_samples_leaf=2, random_state=j),
214     ]
215     return configs[j]
216
217
218 # -----
219 # Model training
220 # -----
221 def train_model(n_samples=1000, test_size=0.2, random_seed=42):
222     """
223     Train one model per target on synthetic Tx line data.
224
225     Returns
226     -----
227     model_bundle : dict
228     """
229     print(f"[ML] Generating {n_samples} synthetic data points...")
230     X, y_raw, y_log = generate_dataset(n_samples=n_samples,
231                                       random_seed=random_seed)
232
233     X_tr, X_te, \
234     y_log_tr, y_log_te, \
235     y_raw_tr, y_raw_te = train_test_split(
236         X, y_log, y_raw,
237         test_size=test_size, random_state=random_seed
238     )
239
240     scaler = StandardScaler()
241     X_tr_s = scaler.fit_transform(X_tr)
242     X_te_s = scaler.transform(X_te)
243
244     models = []
245     r2_scores = []
246     mape_scores = []
247
248     print("[ML] Training one model per target...")
249     for j, name in enumerate(TARGET_NAMES):
250         model = _best_model_for_target(j)
251         model.fit(X_tr_s, y_log_tr[:, j])
252         models.append(model)
253
254         pred_log = model.predict(X_te_s)
255         r2 = r2_score(y_log_te[:, j], pred_log)
256
257         # MAPE in original space
258         if name in ("|Zo|", "alpha", "beta", "VSWR"):
259             y_orig = 10 ** y_log_te[:, j]
260             p_orig = 10 ** pred_log
261         else:      # |Gamma_L|

```

```

262         y_orig = y_log_te[:, j]
263         p_orig = pred_log
264
265         mask = np.abs(y_orig) > 1e-10
266         mape = mean_absolute_percentage_error(
267             y_orig[mask], p_orig[mask]) * 100 if mask.sum() > 0 else 0.0
268
269         r2_scores.append(r2)
270         mape_scores.append(mape)
271
272         algo = "RF" if isinstance(model, RandomForestRegressor) else "GB"
273         print(f" [{j+1}/5] {name:<14} [{algo}] R^2 = {r2:.4f}"
274               f" MAPE = {mape:.2f}%")
275
276     y_pred_all = np.column_stack([m.predict(X_te_s) for m in models])
277     r2_overall = r2_score(y_log_te, y_pred_all)
278
279     print("\n[ML] Training Complete -- Performance on Test Set")
280     print("=" * 55)
281     print(f" {'Target':<14} {'R^2':>8} {'MAPE (%)':>10}")
282     print("-" * 55)
283     for j, name in enumerate(TARGET_NAMES):
284         print(f" {name:<14} {r2_scores[j]:>8.4f} {mape_scores[j]:>10.2f}")
285     print("-" * 55)
286     print(f" {'Overall R^2':<14} {r2_overall:>8.4f}")
287     print("=" * 55)
288
289     if r2_overall >= 0.97:
290         print("[ML] PASS: Target accuracy (R^2 >= 0.97) ACHIEVED.")
291     else:
292         print(f"[ML] INFO: Overall R^2 = {r2_overall:.4f}"
293               f" (target >= 0.97)")
294
295     # Average feature importance
296     fi_list = []
297     for m in models:
298         if hasattr(m, "feature_importances_"):
299             fi_list.append(m.feature_importances_)
300     fi = np.mean(fi_list, axis=0) if fi_list else np.zeros(X.shape[1])
301
302     bundle = {
303         "scaler": scaler,
304         "models": models,
305         "X_test": X_te,
306         "y_log_test": y_log_te,
307         "y_pred_all": y_pred_all,
308         "r2_scores": r2_scores,
309         "mape_scores": mape_scores,
310         "r2_overall": r2_overall,
311         "feature_importances": fi,
312         "feature_names": FEATURE_NAMES,
313         "target_names": TARGET_NAMES,
314     }
315     return bundle
316
317 # -----
318 # Save / Load model
319 # -----
320
321 def save_model(bundle, path=MODEL_PKL):
322     with open(path, "wb") as f:
323         pickle.dump(bundle, f)
324     print(f"[ML] Model saved to: {path}")
325
326
327 def load_model(path=MODEL_PKL):

```

```

328     with open(path, "rb") as f:
329         bundle = pickle.load(f)
330     print(f"[ML] Model loaded from: {path}")
331     return bundle
332
333
334 def get_or_train_model(n_samples=1000, force_retrain=False):
335     """Return trained model bundle. Load from disk or train fresh."""
336     if not force_retrain and os.path.exists(MODEL_PKL):
337         try:
338             return load_model()
339         except Exception:
340             pass
341     bundle = train_model(n_samples=n_samples)
342     save_model(bundle)
343     return bundle
344
345
346 # -----
347 # Prediction interface
348 # -----
349 def predict(bundle, R, L, G, C, f, d, ZL_mag, ZL_angle_deg):
350     """
351     Predict Tx line outputs using the trained per-target models.
352
353     Returns dict with predicted |Zo|, alpha, beta, |Gamma_L|, VSWR.
354     """
355     scaler = bundle["scaler"]
356     models = bundle["models"]
357
358     x = _engineer_features(
359         np.array([R]), np.array([L]), np.array([G]), np.array([C]),
360         np.array([f]), np.array([d]),
361         np.array([ZL_mag]), np.array([ZL_angle_deg])
362     ) # shape (1, 19)
363
364     x_s = scaler.transform(x)
365     preds = [m.predict(x_s)[0] for m in models]
366
367     return {
368         "|Zo|_pred":      10 ** preds[0],
369         "alpha_pred":    10 ** preds[1],
370         "beta_pred":     10 ** preds[2],
371         "|Gamma_L|_pred": np.clip(preds[3], 0.0, 1.0),
372         "VSWR_pred":     max(1.0, 10 ** preds[4]),
373     }
374
375
376 # -----
377 # ML Performance Report Plot
378 # -----
379 def plot_ml_performance(bundle, save=True):
380     """
381     2x3 figure: actual vs predicted for each target + feature importance.
382     """
383     import matplotlib
384     matplotlib.use("Agg")
385     import matplotlib.pyplot as plt
386
387     y_log_te = bundle["y_log_test"]
388     y_pred_all = bundle["y_pred_all"]
389     r2_scores = bundle["r2_scores"]
390     fi = bundle["feature_importances"]
391
392     fig, axes = plt.subplots(2, 3, figsize=(14, 9))
393     axes = axes.flatten()

```

```

394
395 colors = ["#1f77b4", "#ff7f0e", "#2ca02c", "#d62728", "#9467bd"]
396 for j, name in enumerate(TARGET_NAMES):
397     ax = axes[j]
398     ax.scatter(y_log_te[:, j], y_pred_all[:, j],
399               alpha=0.5, s=18, color=colors[j], edgecolors="none")
400     mn = min(y_log_te[:, j].min(), y_pred_all[:, j].min())
401     mx = max(y_log_te[:, j].max(), y_pred_all[:, j].max())
402     ax.plot([mn, mx], [mn, mx], "k--", linewidth=1, label="Ideal")
403     lbl = f"log10({name})" if name not in ("|Gamma_L|",) else name
404     ax.set_title(f"{lbl} (R^2 = {r2_scores[j]:.4f})",
405               fontsize=10, fontweight="bold")
406     ax.set_xlabel("Actual", fontsize=9)
407     ax.set_ylabel("Predicted", fontsize=9)
408     ax.legend(fontsize=7)
409     ax.grid(True, linestyle="--", alpha=0.5)
410     ax.spines["top"].set_visible(False)
411     ax.spines["right"].set_visible(False)
412
413 # Feature importance
414 ax6 = axes[5]
415 fi_arr = np.array(fi)
416 n_top = min(12, len(fi_arr))
417 idx = np.argsort(fi_arr)[-n_top:]
418 fn = bundle["feature_names"]
419 ax6.barh([fn[i] for i in idx], fi_arr[idx],
420          color="#7f7f7f", alpha=0.8)
421 ax6.set_title("Avg. Feature Importances",
422             fontsize=10, fontweight="bold")
423 ax6.set_xlabel("Importance", fontsize=9)
424 ax6.grid(True, axis="x", linestyle="--", alpha=0.5)
425 ax6.spines["top"].set_visible(False)
426 ax6.spines["right"].set_visible(False)
427
428 fig.suptitle(
429     f"ML Model Performance -- RF/GB (per-target)"
430     f" | Overall R^2 = {bundle['r2_overall']:.4f}",
431     fontsize=13, fontweight="bold"
432 )
433 fig.tight_layout()
434
435 if save:
436     from plots import PLOT_DIR
437     path = os.path.join(PLOT_DIR, "08_ml_performance.png")
438     os.makedirs(PLOT_DIR, exist_ok=True)
439     fig.savefig(path, dpi=150, bbox_inches="tight")
440     print(f"[ML] Performance plot saved: {path}")
441
442 return fig
443
444
445 # -----
446 # Stand-alone entry point
447 # -----
448 if __name__ == "__main__":
449     bundle = get_or_train_model(n_samples=1000, force_retrain=True)
450     plot_ml_performance(bundle, save=True)
451     print("\n[ML] Sample prediction:")
452     pred = predict(bundle,
453                   R=10, L=250e-9, G=1e-4, C=100e-12,
454                   f=100e6, d=2.0,
455                   ZL_mag=75, ZL_angle_deg=30)
456     for k, v in pred.items():
457         print(f" {k:<22} = {v:.6g}")

```

D. Tkinter Graphical User Interface (gui.py)

```
1 """
2 gui.py
3 =====
4 Tkinter GUI      Transmission Line Analysis & Design Window
5 =====
6 A complete interactive interface providing:
7 - Input fields for all Tx line parameters
8 - "Compute Analytics" button      shows all derived parameters
9 - "Plot Waveforms" button        renders all 7 plots in a tabbed viewer
10 - "ML Predict" button            shows ML-predicted parameters vs. exact values
11 - "Train/Reload Model" button    (re-)trains the Random Forest model
12 - Status bar with progress messages
13
14 Layout:
15
16     TITLE BAR
17
18     INPUT PANEL                RESULTS PANEL
19     (R, L, G, C, f,           (Zo,      ,  $\alpha$ ,  $\beta$ ,  $\Gamma$ , VSWR, Zin)
20     d, ZL_r, ZL_i)            + ML Predictions
21
22     BUTTON BAR
23
24     PLOT CANVAS (tabbed      7 tabs, one per plot)
25
26     STATUS BAR
27
28 """
29
30 import tkinter as tk
31 from tkinter import ttk, messagebox
32 import threading
33 import numpy as np
34 import os
35
36 import matplotlib
37 matplotlib.use("TkAgg")
38 from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
39 import matplotlib.pyplot as plt
40
41 from analytics import compute_all
42 from plots import generate_all_plots
43 import ml_model as mlm
44
45 # -----
46 # Colour / font constants
47 # -----
48 BG_MAIN      = "#1e1e2e"      # dark background
49 BG_PANEL     = "#2a2a3e"      # slightly lighter panel
50 BG_INPUT     = "#313147"
51 FG_TEXT      = "#cdd6f4"
52 FG_LABEL     = "#89b4fa"
53 FG_VALUE     = "#a6e3a1"
54 FG_ML        = "#f9e2af"
55 FG_TITLE     = "#cba6f7"
56 BTN_BG       = "#45475a"
57 BTN_FG       = "#cdd6f4"
58 BTN_ACTIVE   = "#585b70"
59 ACCENT       = "#89dceb"
60
61 FONT_TITLE   = ("Segoe UI", 14, "bold")
62 FONT_HEADER  = ("Segoe UI", 10, "bold")
63 FONT_LABEL   = ("Segoe UI", 9)
```

```

64 FONT_VALUE = ("Consolas", 9, "bold")
65 FONT_BTN   = ("Segoe UI", 9, "bold")
66 FONT_STATUS = ("Segoe UI", 8)
67
68 # -----
69 # Default parameter values (coaxial-cable-like example)
70 # -----
71 DEFAULTS = {
72     "R":      "5.0",          #  $\Omega/m$ 
73     "L":      "250e-9",       # H/m
74     "G":      "1e-4",         # S/m
75     "C":      "100e-12",      # F/m
76     "f":      "100e6",        # Hz    (100 MHz)
77     "d":      "2.0",          # m
78     "ZL_r":   "75.0",         #  $\Omega$  (real part)
79     "ZL_i":   "30.0",         #  $\Omega$  (imaginary part)
80     "f_start": "1e6",         # Hz (freq sweep start)
81     "f_end":  "1e9",          # Hz (freq sweep end)
82 }
83
84 # -----
85 # Helper: formatted row in results panel
86 # -----
87 def _fmt(val, unit="", is_complex=False):
88     if is_complex:
89         r, i = np.real(val), np.imag(val)
90         sign = "+" if i >= 0 else "-"
91         return f"{r:.4f} {sign} j{abs(i):.4f} {unit}"
92     if isinstance(val, (float, np.floating)):
93         return f"{val:.6g} {unit}"
94     return f"{val} {unit}"
95
96 # -----
97 # Main Application Window
98 # -----
99
100 class TxLineApp(tk.Tk):
101     def __init__(self):
102         super().__init__()
103         self.title("Transmission Line Analyzer      Engineering Electromagnetics")
104         self.configure(bg=BG_MAIN)
105         self.resizable(True, True)
106         self.geometry("1280x820")
107         self.minsize(1100, 700)
108
109         # Model bundle (loaded lazily)
110         self._bundle = None
111
112         self._build_ui()
113         self.after(200, self._load_model_async)
114
115     # -----
116     # UI construction
117     # -----
118     def _build_ui(self):
119         """Build all UI sections."""
120         self._build_title()
121         self._build_main_frame()
122         self._build_input_panel()
123         self._build_results_panel()
124         self._build_button_bar()
125         self._build_plot_area()
126         self._build_status_bar()
127
128     def _build_title(self):
129         frm = tk.Frame(self, bg=BG_MAIN)

```

```

130         frm.pack(fill="x", padx=16, pady=(12, 0))
131     tk.Label(frm,
132             text="          Transmission Line Analysis & Design Window",
133             font=FONT_TITLE, bg=BG_MAIN, fg=FG_TITLE).pack(side="left")
134     tk.Label(frm,
135             text="Engineering Electromagnetics | Module-1",
136             font=FONT_LABEL, bg=BG_MAIN, fg=FG_TEXT).pack(side="right")
137
138     def _build_main_frame(self):
139         self._main_frm = tk.Frame(self, bg=BG_MAIN)
140         self._main_frm.pack(fill="x", padx=12, pady=6)
141
142     def _build_input_panel(self):
143         """Left panel: parameter input fields."""
144         panel = tk.LabelFrame(self._main_frm, text="          Input Parameters   ",
145                               bg=BG_PANEL, fg=FG_LABEL,
146                               font=FONT_HEADER, bd=1, relief="groove",
147                               padx=10, pady=8)
148         panel.pack(side="left", fill="y", padx=(0, 6))
149
150         self._entries = {}
151
152         fields = [
153             ("R  ( $\Omega$ /m)", "R", "Resistance per unit length"),
154             ("L  (H/m)", "L", "Inductance per unit length"),
155             ("G  (S/m)", "G", "Conductance per unit length"),
156             ("C  (F/m)", "C", "Capacitance per unit length"),
157             ("f  (Hz)", "f", "Frequency"),
158             ("d  (m)", "d", "Line length / distance from load"),
159             ("ZL_real ( $\Omega$ )", "ZL_r", "Real part of load impedance"),
160             ("ZL_imag ( $\Omega$ )", "ZL_i", "Imaginary part of load impedance"),
161             ("f_start (Hz)", "f_start", "Frequency sweep start"),
162             ("f_end  (Hz)", "f_end", "Frequency sweep end"),
163         ]
164
165         for row_i, (label, key, tip) in enumerate(fields):
166             tk.Label(panel, text=label, bg=BG_PANEL, fg=FG_LABEL,
167                     font=FONT_LABEL, anchor="w", width=18).grid(
168                 row=row_i, column=0, sticky="w", pady=3)
169             var = tk.StringVar(value=DEFAULTS.get(key, ""))
170             ent = tk.Entry(panel, textvariable=var, width=14,
171                           bg=BG_INPUT, fg=FG_TEXT,
172                           insertbackground=FG_TEXT,
173                           font=FONT_VALUE, bd=0, relief="flat")
174             ent.grid(row=row_i, column=1, padx=(6, 0), pady=3)
175             self._entries[key] = var
176
177     def _build_results_panel(self):
178         """Right panel: computed results display."""
179         panel = tk.LabelFrame(self._main_frm, text="          Computed Parameters   ",
180                               bg=BG_PANEL, fg=FG_LABEL,
181                               font=FONT_HEADER, bd=1, relief="groove",
182                               padx=10, pady=8)
183         panel.pack(side="left", fill="both", expand=True)
184
185         # Analytical results
186         tk.Label(panel, text="          Analytical Results   ",
187                 bg=BG_PANEL, fg=ACCENT,
188                 font=FONT_HEADER).grid(row=0, column=0, columnspan=2,
189                                         sticky="w", pady=(0, 4))
190
191         self._result_vars = {}
192         result_fields = [
193             ("Zo", "|Zo|", " $\Omega$ "),
194             (" Zo ", "angle_Zo_deg", " $^\circ$ "),
195             (" (alpha + j $\beta$ )", "gamma_str", ""),

```

```

196         ("α (attenuation)", "alpha", "Np/m"),
197         ("β (phase)", "beta", "rad/m"),
198         ("Γ at load", "|Gamma_L|", ""),
199         ("Γ", "angle_Gamma_deg", "°"),
200         ("VSWR", "VSWR", ""),
201         ("Zin", "|Zin|", "Ω"),
202         ("Zin ", "angle_Zin_deg", "°"),
203     ]
204
205     for row_i, (label, key, unit) in enumerate(result_fields):
206         tk.Label(panel, text=label, bg=BG_PANEL, fg=FG_LABEL,
207                 font=FONT_LABEL, anchor="w", width=22).grid(
208             row=row_i+1, column=0, sticky="w", pady=2)
209         var = tk.StringVar(value=" ")
210         tk.Label(panel, textvariable=var, bg=BG_PANEL, fg=FG_VALUE,
211                 font=FONT_VALUE, anchor="w", width=30).grid(
212             row=row_i+1, column=1, sticky="w", padx=6)
213         self._result_vars[key] = var
214
215     # Divider
216     ttk.Separator(panel, orient="horizontal").grid(
217         row=len(result_fields)+2, column=0, columnspan=2,
218         sticky="ew", pady=8)
219
220     # ML results
221     tk.Label(panel, text=" ML Predictions (Random Forest) ",
222             bg=BG_PANEL, fg=FG_ML,
223             font=FONT_HEADER).grid(
224         row=len(result_fields)+3, column=0, columnspan=2,
225         sticky="w", pady=(0, 4))
226
227     self._ml_vars = {}
228     ml_fields = [
229         ("|Zo| predicted", "|Zo|_pred", "Ω"),
230         ("α predicted", "alpha_pred", "Np/m"),
231         ("β predicted", "beta_pred", "rad/m"),
232         ("|Γ| predicted", "|Gamma_L|_pred", ""),
233         ("VSWR predicted", "VSWR_pred", ""),
234         ("Model R (overall)", "r2_overall", ""),
235     ]
236
237     base = len(result_fields) + 4
238     for row_i, (label, key, unit) in enumerate(ml_fields):
239         tk.Label(panel, text=label, bg=BG_PANEL, fg=FG_ML,
240                 font=FONT_LABEL, anchor="w", width=22).grid(
241             row=base + row_i, column=0, sticky="w", pady=2)
242         var = tk.StringVar(value=" ")
243         tk.Label(panel, textvariable=var, bg=BG_PANEL, fg=FG_ML,
244                 font=FONT_VALUE, anchor="w", width=30).grid(
245             row=base + row_i, column=1, sticky="w", padx=6)
246         self._ml_vars[key] = var
247
248     def _build_button_bar(self):
249         """Action buttons."""
250         bar = tk.Frame(self, bg=BG_MAIN)
251         bar.pack(fill="x", padx=12, pady=4)
252
253         btn_cfg = dict(bg=BTN_BG, fg=BTN_FG, font=FONT_BTN,
254                       activebackground=BTN_ACTIVE, activeforeground=FG_TEXT,
255                       bd=0, padx=16, pady=6, cursor="hand2", relief="flat")
256
257         tk.Button(bar, text=" Compute Analytics",
258                 command=self._compute, **btn_cfg).pack(side="left", padx=4)
259         tk.Button(bar, text=" Plot All Waveforms",
260                 command=self._plot_all, **btn_cfg).pack(side="left", padx=4)
261         tk.Button(bar, text=" ML Predict",

```

```

262         command=self._ml_predict, **btn_cfg).pack(side="left", padx=4)
263     tk.Button(bar, text="        Train / Reload Model",
264               command=self._train_model, **btn_cfg).pack(side="left", padx=4)
265     tk.Button(bar, text="        Reset Defaults",
266               command=self._reset_defaults, **btn_cfg).pack(side="left", padx=4)
267
268     def _build_plot_area(self):
269         """Tabbed plot canvas area."""
270         self._nb = ttk.Notebook(self)
271         self._nb.pack(fill="both", expand=True, padx=12, pady=(0, 4))
272
273         # Style the notebook
274         style = ttk.Style()
275         style.theme_use("clam")
276         style.configure("TNotebook",
277                         background=BG_MAIN, borderwidth=0)
278         style.configure("TNotebook.Tab",
279                         background=BTN_BG, foreground=FG_TEXT,
280                         font=FONT_LABEL, padding=(8, 4))
281         style.map("TNotebook.Tab",
282                  background=[("selected", BG_PANEL)],
283                  foreground=[("selected", ACCENT)])
284
285         self._tab_frames = {}
286         self._tab_canvases = {}
287
288         tab_labels = [
289             "Voltage |V(z)|",
290             "Current |I(z)|",
291             "Reflection |Γ(z)|",
292             "Zin vs z",
293             "VSWR vs f",
294             "α & β vs f",
295             "Standing Wave",
296             "Loss Behavior",
297             "3D Wave Visualizations",
298             "Impedance Matching",
299             "ML Performance",
300         ]
301         for label in tab_labels:
302             frm = tk.Frame(self._nb, bg=BG_PANEL)
303             self._nb.add(frm, text=f" {label} ")
304             self._tab_frames[label] = frm
305
306         # Placeholder labels in each tab
307         for label, frm in self._tab_frames.items():
308             tk.Label(frm, text=f"[ {label}      click 'Plot All Waveforms' ]",
309                     bg=BG_PANEL, fg=FG_TEXT, font=FONT_LABEL).pack(
310                         expand=True)
311
312     def _build_status_bar(self):
313         self._status_var = tk.StringVar(value="Ready. Enter parameters and click Compute.")
314         bar = tk.Frame(self, bg="#11111b", bd=1, relief="sunken")
315         bar.pack(fill="x", side="bottom")
316         tk.Label(bar, textvariable=self._status_var,
317                 bg="#11111b", fg="#6c7086",
318                 font=FONT_STATUS, anchor="w", padx=8).pack(fill="x")
319
320     # -----
321     # Utility
322     # -----
323     def _status(self, msg):
324         self._status_var.set(msg)
325         self.update_idletasks()
326
327     def _get_params(self):

```

```
328         """Parse all entry fields and return a dict of floats."""
329         try:
330             p = {k: float(v.get()) for k, v in self._entries.items()}
331             return p
332         except ValueError as e:
333             messagebox.showerror("Input Error",
334                                 f"Invalid parameter value:\n{e}")
335             return None
336
337     def _reset_defaults(self):
338         for k, var in self._entries.items():
339             var.set(DEFAULTS.get(k, ""))
340         self._status("Parameters reset to defaults.")
341
342     # -----
343     # Compute Analytics
344     # -----
345     def _compute(self):
346         p = self._get_params()
347         if p is None:
348             return
349         self._status("Computing analytical parameters...")
350         try:
351             res = compute_all(p["R"], p["L"], p["G"], p["C"],
352                             p["f"], p["d"], p["ZL_r"], p["ZL_i"])
353
354             # Format gamma as string
355             a, b = res["alpha"], res["beta"]
356             sign = "+" if b >= 0 else "-"
357             gamma_str = f"{a:.4e} {sign} j{abs(b):.4e}"
358
359             self._result_vars["|Zo|"].set(_fmt(res["|Zo|"], "Ω"))
360             self._result_vars["angle_Zo_deg"].set(_fmt(res["angle_Zo_deg"], "°"))
361             self._result_vars["gamma_str"].set(gamma_str)
362             self._result_vars["alpha"].set(_fmt(res["alpha"], "Np/m"))
363             self._result_vars["beta"].set(_fmt(res["beta"], "rad/m"))
364             self._result_vars["|Gamma_L|"].set(_fmt(res["|Gamma_L|"], ""))
365             self._result_vars["angle_Gamma_deg"].set(
366                 _fmt(res["angle_Gamma_deg"], "°"))
367             self._result_vars["VSWR"].set(_fmt(res["VSWR"], ""))
368             self._result_vars["|Zin|"].set(_fmt(res["|Zin|"], "Ω"))
369             self._result_vars["angle_Zin_deg"].set(
370                 _fmt(res["angle_Zin_deg"], "°"))
371
372             self._status("    Analytics computed successfully.")
373         except Exception as e:
374             messagebox.showerror("Computation Error", str(e))
375             self._status("Error during computation.")
376
377     # -----
378     # Plot All Waveforms
379     # -----
380     def _plot_all(self):
381         p = self._get_params()
382         if p is None:
383             return
384         self._status("Generating all 10 analytical plots      please wait...")
385         self._compute() # refresh analytics first
386
387         def _worker():
388             try:
389                 figs = generate_all_plots(
390                     p["R"], p["L"], p["G"], p["C"],
391                     p["f"], p["d"], p["ZL_r"], p["ZL_i"],
392                     f_start=p["f_start"], f_end=p["f_end"],
393                     save=True
```

```
394         )
395         self.after(0, lambda: self._render_plots(figs))
396     except Exception as e:
397         self.after(0, lambda: messagebox.showerror(
398             "Plot Error", str(e)))
399         self.after(0, lambda: self._status("Plot generation failed.))
400
401     threading.Thread(target=_worker, daemon=True).start()
402
403 def _render_plots(self, figs):
404     """Embed matplotlib figures into Notebook tabs."""
405     tab_labels = [
406         "Voltage |V(z)|",
407         "Current |I(z)|",
408         "Reflection |Γ(z)|",
409         "Zin vs z",
410         "VSWR vs f",
411         " $\alpha$  &  $\beta$  vs f",
412         "Standing Wave",
413         "Loss Behavior",
414         "3D Wave Visualizations",
415         "Impedance Matching",
416     ]
417     for (title, fig), tab_label in zip(figs, tab_labels):
418         frm = self._tab_frames[tab_label]
419         # Clear old widgets
420         for w in frm.wininfo_children():
421             w.destroy()
422
423         canvas = FigureCanvasTkAgg(fig, master=frm)
424         canvas.draw()
425         canvas.get_tk_widget().pack(fill="both", expand=True)
426         self._tab_c canvases[tab_label] = canvas
427
428     self._status("    All 10 analytical plots generated and saved to plots_output/")
429
430     # Also try ML performance plot if model is ready
431     if self._bundle is not None:
432         try:
433             fig_ml = mlm.plot_ml_performance(self._bundle, save=True)
434             frm = self._tab_frames["ML Performance"]
435             for w in frm.wininfo_children():
436                 w.destroy()
437             canvas = FigureCanvasTkAgg(fig_ml, master=frm)
438             canvas.draw()
439             canvas.get_tk_widget().pack(fill="both", expand=True)
440             plt.close(fig_ml)
441         except Exception:
442             pass
443
444     for fig in [f for _, f in figs]:
445         plt.close(fig)
446
447     # -----
448     # ML Predict
449     # -----
450 def _ml_predict(self):
451     if self._bundle is None:
452         messagebox.showinfo("Model Not Ready",
453             "ML model is still loading. Please wait a moment.")
454     return
455     p = self._get_params()
456     if p is None:
457         return
458     self._status("Running ML prediction...")
459     try:
```

```

460         ZL = complex(p["ZL_r"], p["ZL_i"])
461         ZL_mag = abs(ZL)
462         ZL_angle = np.degrees(np.angle(ZL))
463
464         pred = mlm.predict(
465             self._bundle,
466             R=p["R"], L=p["L"], G=p["G"], C=p["C"],
467             f=p["f"], d=p["d"],
468             ZL_mag=ZL_mag, ZL_angle_deg=ZL_angle
469         )
470
471         self._ml_vars["|Zo|_pred"].set(
472             f"{pred['|Zo|_pred']:.4f}  $\Omega$ ")
473         self._ml_vars["alpha_pred"].set(
474             f"{pred['alpha_pred']:.4e} Np/m")
475         self._ml_vars["beta_pred"].set(
476             f"{pred['beta_pred']:.4e} rad/m")
477         self._ml_vars["|Gamma_L|_pred"].set(
478             f"{pred['|Gamma_L|_pred']:.4f}")
479         self._ml_vars["VSWR_pred"].set(
480             f"{pred['VSWR_pred']:.4f}")
481         self._ml_vars["r2_overall"].set(
482             f"{self._bundle['r2_overall']:.4f}")
483
484         self._status("ML prediction complete.")
485     except Exception as e:
486         messagebox.showerror("ML Error", str(e))
487         self._status("ML prediction failed.")
488
489     # -----
490     # Train / Reload Model
491     # -----
492     def _train_model(self):
493         self._status("Training ML model (500 samples)      please wait...")
494
495         def _worker():
496             try:
497                 bundle = mlm.get_or_train_model(n_samples=1000,
498                                                 force_retrain=True)
499
500                 self._bundle = bundle
501                 r2 = bundle['r2_overall']
502                 status = "PASS" if r2 >= 0.97 else "INFO"
503                 self.after(0, lambda: self._status(
504                     f"[{status}] Model trained. R^2 = {r2:.4f}"))
505             except Exception as e:
506                 self.after(0, lambda: messagebox.showerror(
507                     "Training Error", str(e)))
508                 self.after(0, lambda: self._status("Model training failed.))
509
510         threading.Thread(target=_worker, daemon=True).start()
511
512     def _load_model_async(self):
513         """Silently load (or train) model in background on startup."""
514         def _worker():
515             try:
516                 self._bundle = mlm.get_or_train_model(n_samples=1000,
517                                                         force_retrain=False)
518
519                 r2 = self._bundle['r2_overall']
520                 self.after(0, lambda: self._status(
521                     f"ML model ready. R^2 = {r2:.4f} -- Enter parameters and click
522                     Compute.))
523             except Exception as e:
524                 self.after(0, lambda: self._status(
525                     f"ML model load failed: {e}. Use 'Train/Reload Model' button.))
526
527         threading.Thread(target=_worker, daemon=True).start()

```

```
525  
526  
527 if __name__ == "__main__":  
528     app = TxLineApp()  
529     app.mainloop()
```

E. Application Launcher and CLI Interface (main.py)

```
1 """
2 main.py
3 =====
4 Transmission Line Analyzer      Entry Point
5 =====
6 Engineering Electromagnetics | Module-1 Assignment
7
8 Usage:
9     python main.py [--cli] [--train] [--plots]
10
11 Flags:
12     (no flags)    Launch the full GUI application
13     --cli         Run a command-line demonstration (no GUI)
14     --train       Force retrain the ML model and exit
15     --plots       Generate all plots for default parameters and exit
16
17 Author   : Student | Engineering Electromagnetics
18 Course   : Transmission Lines      Module 1
19 """
20
21 import sys
22 import os
23 import argparse
24 import numpy as np
25
26 # Ensure the project directory is in PYTHONPATH
27 sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
28
29
30 # -----
31 # CLI demonstration (no GUI, no Tkinter dependency)
32 # -----
33 def run_cli_demo():
34     """
35     Run a full command-line demonstration:
36     1. Compute all analytical parameters for a sample Tx line
37     2. Train / load the ML model
38     3. Compare analytical vs. ML-predicted values
39     4. Print a summary table
40     """
41     from analytics import compute_all
42     from ml_model import get_or_train_model, predict
43
44     print("\n" + "=" * 60)
45     print("  TRANSMISSION LINE ANALYZER -- CLI Demo")
46     print("  Engineering Electromagnetics | Module-1")
47     print("=" * 60)
48
49     #      Example parameters
50
51     params = dict(
52         R      = 5.0,          #  $\Omega/m$ 
53         L      = 250e-9,       #  $H/m$ 
54         G      = 1e-4,         #  $S/m$ 
55         C      = 100e-12,      #  $F/m$ 
56         f      = 100e6,        #  $Hz$  (100 MHz)
57         d      = 2.0,          #  $m$ 
58         ZL_real = 75.0,         #  $\Omega$       must match compute_all() signature
59         ZL_imag = 30.0,         #  $\Omega$ 
60     )
61
62     print("\n[1] INPUT PARAMETERS")
63     print("-" * 40)
```

```

63     for k, v in params.items():
64         print(f"    {k:<8} = {v:.4g}")
65
66     #           Analytical computation
67
68     print("\n[2] ANALYTICAL RESULTS")
69     print("-" * 40)
70     res = compute_all(**params)
71
72     a, b = res["alpha"], res["beta"]
73     sign = "+" if b >= 0 else "-"
74     print(f"    {'Zo':<20} = {res['|Zo|']:.4f} Ohm /{res['angle_Zo_deg']:.2f} deg")
75     print(f"    {'gamma':<20} = {a:.4e} {sign} j{abs(b):.4e} 1/m")
76     print(f"    {'alpha':<20} = {res['alpha']:.6e} Np/m")
77     print(f"    {'beta':<20} = {res['beta']:.6e} rad/m")
78     print(f"    {'|Gamma_L|':<20} = {res['|Gamma_L|']:.6f} "
79           f"/{res['angle_Gamma_deg']:.2f} deg")
80     print(f"    {'VSWR':<20} = {res['VSWR']:.4f}")
81     print(f"    {'Zin':<20} = {res['|Zin|']:.4f} Ohm /{res['angle_Zin_deg']:.2f} deg")
82
83     # -- ML model -----
84     print("\n[3] ML MODEL -- Random Forest Regressor")
85     print("-" * 40)
86     bundle = get_or_train_model(n_samples=500, force_retrain=False)
87     print(f"    Overall R^2 (test set) = {bundle['r2_overall']:.4f}")
88     print(f"    Status: {'PASS (>=0.97)' if bundle['r2_overall'] >= 0.97 else 'FAIL (<0.97)'}")
89
90     print("\n    Per-target R^2 scores:")
91     for name, r2, mape in zip(bundle["target_names"],
92                               bundle["r2_scores"],
93                               bundle["mape_scores"]):
94         print(f"        {name:<14} R^2 = {r2:.4f} MAPE = {mape:.2f}%")
95
96     #           Compare analytical vs ML
97
98     print("\n[4] ANALYTICAL vs. ML PREDICTION")
99     print("-" * 60)
100     ZL = complex(params["ZL_real"], params["ZL_imag"])
101     pred = predict(bundle,
102                   R=params["R"], L=params["L"],
103                   G=params["G"], C=params["C"],
104                   f=params["f"], d=params["d"],
105                   ZL_mag=abs(ZL),
106                   ZL_angle_deg=np.degrees(np.angle(ZL)))
107
108     comparisons = [
109         ("|Zo| (Ohm)", res["|Zo|"], pred["|Zo|_pred"]),
110         ("alpha (Np/m)", res["alpha"], pred["alpha_pred"]),
111         ("beta (rad/m)", res["beta"], pred["beta_pred"]),
112         ("|Gamma_L|", res["|Gamma_L|"], pred["|Gamma_L|_pred"]),
113         ("VSWR", res["VSWR"], pred["VSWR_pred"]),
114     ]
115
116     print(f"    {'Parameter':<18} {'Analytical':>14} {'ML Predicted':>14} {'Error%':>9}")
117     print("    " + "-" * 58)
118     for name, exact, ml_val in comparisons:
119         err = abs(exact - ml_val) / (abs(exact) + 1e-12) * 100
120         print(f"    {name:<18} {exact:>14.6g} {ml_val:>14.6g} {err:>8.2f}%")
121
122     #           Generate plots
123
124     print("\n[5] GENERATING ALL 10 PLOTS (saved to plots_output/)"
125     print("-" * 40)
126     try:
127         from plots import generate_all_plots
128         import matplotlib.pyplot as plt

```

```

126     figs = generate_all_plots(
127         params["R"], params["L"], params["G"], params["C"],
128         params["f"], params["d"], params["ZL_real"], params["ZL_imag"],
129         f_start=1e6, f_end=1e9, save=True
130     )
131     for title, fig in figs:
132         plt.close(fig)
133     from plots import PLOT_DIR
134     print(f"    [OK] All plots saved in: {PLOT_DIR}")
135
136     # ML performance plot
137     from ml_model import plot_ml_performance
138     fig_ml = plot_ml_performance(bundle, save=True)
139     plt.close(fig_ml)
140 except Exception as e:
141     print(f"    Plot generation error: {e}")
142
143     print("\n" + "=" * 60)
144     print("    CLI Demo Complete.")
145     print("=" * 60 + "\n")
146
147
148 # -----
149 # GUI launcher
150 # -----
151 def run_gui():
152     """Launch the Tkinter GUI application."""
153     try:
154         from gui import TxLineApp
155         app = TxLineApp()
156         app.mainloop()
157     except Exception as e:
158         print(f"[ERROR] GUI failed to start: {e}")
159         print("Try running with --cli flag for command-line mode.")
160         sys.exit(1)
161
162
163 # -----
164 # Entry point
165 # -----
166 def main():
167     parser = argparse.ArgumentParser(
168         description="Transmission Line Analyzer      Engineering Electromagnetics"
169     )
170     parser.add_argument("--cli", action="store_true",
171                         help="Run CLI demonstration (no GUI)")
172     parser.add_argument("--train", action="store_true",
173                         help="Force retrain ML model and exit")
174     parser.add_argument("--plots", action="store_true",
175                         help="Generate plots for default parameters and exit")
176
177     args = parser.parse_args()
178
179     if args.train:
180         from ml_model import get_or_train_model, plot_ml_performance
181         import matplotlib.pyplot as plt
182         bundle = get_or_train_model(n_samples=500, force_retrain=True)
183         fig = plot_ml_performance(bundle, save=True)
184         plt.close(fig)
185         print("Model training complete. Exiting.")
186         return
187
188     if args.plots:
189         from plots import generate_all_plots
190         import matplotlib.pyplot as plt
191         figs = generate_all_plots(

```

```
192         5.0, 250e-9, 1e-4, 100e-12,
193         100e6, 2.0, 75.0, 30.0,
194         f_start=1e6, f_end=1e9, save=True
195     )
196     for _, fig in figs:
197         plt.close(fig)
198     from plots import PLOT_DIR
199     print(f"Plots saved in: {PLOT_DIR}")
200     return
201
202     if args.cli:
203         run_cli_demo()
204         return
205
206     # Default: launch GUI
207     run_gui()
208
209
210 if __name__ == "__main__":
211     main()
```